Optimization Log Copilot: Using Large Language Models to Analyze Mathematical Optimization Solver Logs

Jeff Ginn
EN.705.605 Introduction to Generative AI

May 2, 2026

Abstract

Modern optimization solvers for linear programming (LP) and mixed-integer programming (MIP) produce detailed logs describing solver progress, including presolve reductions, cutting plane generation, branching decisions, heuristic activity, and bound progression. Interpreting these logs to diagnose performance bottlenecks requires significant domain expertise and manual effort, limiting accessibility for many practitioners. This paper presents the Optimization Log Copilot, a system that leverages large language models (LLMs) to automatically analyze solver logs and generate structured diagnostic summaries. The proposed approach combines structured log parsing with transformer-based sequence understanding and generative language models to perform three tasks: log summarization, performance issue detection, and solver recommendation generation. We describe the system architecture, a dataset of approximately 200 solver logs collected from MIPLIB 2017 benchmarks and Ramsey-type problems from the Rose solver, and a multi-label classification framework for bottleneck detection. Preliminary results from prototype development demonstrate the feasibility of structured log parsing and rule-based diagnostic generation, with planned experiments to evaluate LLM-based approaches using metrics including precision, recall, F1 score, ROUGE, and human expert evaluation.

1 Introduction

Modern optimization solvers for linear programming (LP) and mixed-integer linear programming (MILP) produce logs that describe solver progress in detail, including presolve activity, cut generation, heuristic behavior, branching decisions, node exploration, optimality gap progression, and termination status. While these logs contain valuable diagnostic information, they are difficult to interpret without deep expertise in optimization algorithms. As a result, users often struggle to understand why a solver run is slow, fails to converge, or exhibits unstable behavior. Diagnosing these issues typically requires manual inspection, which limits accessibility and productivity.

The difficulty of solver log analysis stems from several factors. First, solver logs are semi-structured text that mix numerical output with free-form status messages. Second, the logs can be extremely long, sometimes containing thousands of lines for a single solve. Third, the terminology and metrics are domain-specific: concepts such as LP relaxation gaps, simplex pivots, Gomory cuts, and branching pseudocosts require familiarity with mathematical optimization theory. Finally, the patterns that indicate performance issues are often subtle and context-dependent. For example, a large root LP gap alone may or may not indicate a weak formulation, depending on the problem structure and subsequent bound improvement.

Recent advances in large language models (LLMs) have demonstrated strong capabilities in processing semi-structured text, including system log analysis for incident detection [1], anomaly

detection in software infrastructure [2], and reasoning over structured data. However, these techniques have been applied primarily to IT system logs and software engineering contexts, not to mathematical optimization solver logs.

This project explores whether LLMs can serve as an "Optimization Log Copilot" that automatically analyzes solver logs and provides diagnostic insights about solver performance. The system aims to answer questions such as: Why is the solver exploring a large number of nodes? Is the root LP relaxation weak? Are cutting planes improving the bound? What structural issues might exist in the optimization model?

The proposed approach combines structured log parsing with transformer-based sequence understanding and conditional text generation. The system is designed as a post-processing advisory layer that operates independently of solver internals. The ML problems addressed include multi-label classification for bottleneck detection and sequence-to-sequence generation for diagnostic explanation.

The remainder of this paper is organized as follows. Section 2 surveys related work in LLM-based log analysis, deep learning for anomaly detection, and machine learning for combinatorial optimization. Section 3 defines the research problem and hypotheses. Section 4 describes the proposed method, including the system architecture, dataset, and training approach. Section 5 outlines the planned experimental evaluation. Section 6 presents results. Section 7 discusses conclusions and future directions.

2 Related Work

Several research areas inform this project: LLM-based log analysis, deep learning for anomaly detection in log data, and machine learning for combinatorial optimization.

2.1 LLM-Based Log Analysis

Akhtar et al. [1] survey how large language models are being applied to event log analysis, particularly in system reliability and software engineering contexts. They identify that traditional approaches to log analysis require heavy feature engineering, depend on rigid log parsing, and struggle with unstructured or evolving log formats. The survey categorizes LLM-based approaches into four groups: fine-tuning, in-context learning, retrieval-augmented generation (RAG), and hybrid symbolic–LLM systems. A key finding is that RAG is strongly preferred over fine-tuning for log analysis because fine-tuning LLMs on logs is expensive and brittle, while RAG performs well with smaller datasets and reduces hallucination risk. The survey also notes that log analysis evaluation often lacks ground truth and that human judgment remains necessary, suggesting that an LLM-based copilot should supplement rather than replace human reasoning.

2.2 Deep Learning for Anomaly Detection in Log Data

Landauer et al. [2] survey deep learning methods for log anomaly detection, covering system failures, incidents, intrusions, and operational breakdowns. The methods reviewed include sequence-based models (LSTMs, GRUs, and Transformer architectures), reconstruction-based models (autoencoders and variational autoencoders), and graph-based methods. Key findings include that transformer models outperform LSTMs in modeling long-range dependencies, and that deep models generally outperform classical statistical anomaly detectors, with reported F1 scores ranging from 0.90 to 0.98 for deep LSTM-based models and often exceeding 0.95 for transformer-based approaches. Common evaluation metrics include precision, recall, F1 score, and ROC-AUC. However,

these systems have been applied primarily to software infrastructure logs rather than optimization solver logs, leaving the domain of solver diagnostics largely unexplored.

2.3 Machine Learning for Combinatorial Optimization

Bengio et al. [3] survey how machine learning can assist combinatorial optimization solvers by learning decision policies. They categorize ML usage into learning to configure (parameter tuning, solver selection), learning to guide search (branching policies, cut selection, node selection), learning surrogate models (predicting solver runtime and node counts), and reinforcement learning for search policies. The survey reports that learned branching strategies can yield 10–30% runtime reductions over handcrafted heuristics, and that ML-based solver configuration can reduce runtime significantly. For this project, the survey underscores that ML-driven solver diagnostics and decision support are important research directions, particularly for recommending parameter settings such as cut aggressiveness, heuristic configurations, and presolve adjustments based on log analysis.

Kotary et al. [4] survey end-to-end constrained optimization learning, providing additional context on how ML models can be integrated with optimization solvers. Yazdani et al. [5] demonstrate how language models can be used to strengthen integer programs through evolution-guided cut generation, illustrating a direct connection between LLMs and optimization solver improvement.

2.4 Gap in the Literature

While LLM-based log analysis has been studied extensively for IT infrastructure and software systems, and machine learning has been applied to various aspects of optimization solver control, using LLMs to analyze and diagnose optimization solver logs remains largely unexplored. Optimization researchers typically analyze solver logs manually through ad hoc inspection of branch-and-bound traces, cut effectiveness metrics, and solver benchmarking output. Automated interpretation tools for solver logs are rare. This project addresses that gap.

3 Research Problem and Hypotheses

3.1 Problem Description

Solver logs are semi-structured sequences containing solver events, numerical metrics, and domain-specific terminology. Modern solvers internally adapt their behavior based on problem structure adjusting branching strategies, selecting cuts, and configuring heuristics, but this information is encoded in log output that is not easily accessible to users. The research problem addressed in this project is:

> *Given a raw solver log from a LP/MIP solver, can a machine learning system generate a structured diagnostic summary and identify likely performance bottlenecks?*

Typical performance issues that the system should detect include:

- Weak LP relaxation: Large initial optimality gap at the root node, indicating the LP relaxation provides a poor bound.
- Excessive branching: Exploding node counts during branch-and-bound search, often caused by weak variable selection or symmetry.
- Ineffective cut generation: Many cutting plane rounds with little bound improvement.

- Degeneracy: Slow LP pivots and numerical instability during the simplex method.
- Insufficient presolve: Minimal problem reduction during the presolve phase.

3.2 Research Hypotheses

The project investigates three research questions, formalized as testable hypotheses:

- H1 (Summarization): An LLM-based system can generate accurate natural-language summaries of solver log behavior, as measured by ROUGE scores and human evaluation against expert-written summaries.
- H2 (Bottleneck Detection): Solver logs contain sufficient information to classify the cause of poor solver performance, and a trained classifier can achieve F1 scores comparable to those reported in related log anomaly detection work (>0.85).
- H3 (Recommendations): The system can produce actionable solver configuration recommendations that correlate with measurable improvements in solver performance metrics such as solve time and node count.

4 Method

The Optimization Log Copilot combines structured log parsing, transformer-based sequence understanding, and generative language models to produce both structured predictions and natural-language explanations. The system does not modify solver internals.

4.1 System Architecture

The system pipeline consists of four stages, shown in Figure 1.

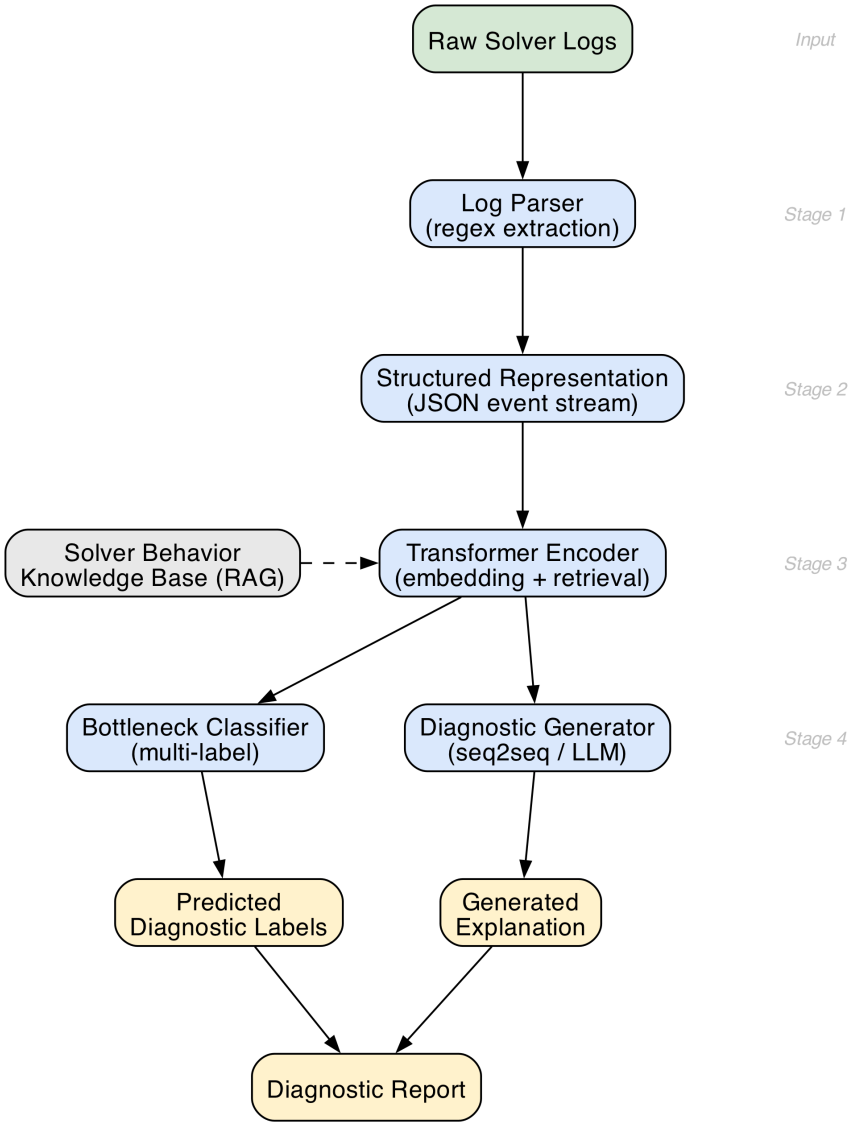

Figure 1: Optimization Log Copilot system architecture. Raw solver logs are parsed into structured JSON events, encoded via a transformer, and processed by: a multi-label bottleneck classifier and a seq2seq diagnostic generator. An RAG knowledge base augments the encoder with known solver behavior patterns.

1. Log Parsing: Raw solver log text is parsed into a structured event stream. A regex-based parser extracts key fields including model dimensions, presolve statistics, root relaxation objective, node-by-node progress (incumbent, best bound, gap), cutting plane counts by type, heuristic solutions found, and termination status.

2. Structured Representation: Parsed events are converted into a JSON-based structured format that captures the progression of the solve. Each log is represented as a sequence of events with associated metrics, enabling downstream models to reason over solver progress more effectively than using raw text alone.

3. Embedding and Retrieval: Structured log representations are embedded using a transformer

encoder.

4. LLM Reasoning and Generation: The language model receives the structured log representation (and optionally retrieved examples) and produces a diagnostic report containing a summary of solver behavior, predicted bottleneck labels, and recommended actions.

4.2 Dataset Construction

The dataset consists of approximately 200 solver logs collected from two sources:

- MIPLIB 2017 benchmarks: Standard benchmark instances from the Mixed Integer Programming Library (`https://miplib.zib.de/`), representing a diverse set academic MIP problems.
- Ramsey-type problems: A series of combinatorial models representing Ramsey problems (e.g., $R(4, 4; 15)$), which exhibit a range of solver behaviors including symmetry, large search trees, infeasibility, and difficult relaxations.

Logs have been generated using the Rose solver. An automated benchmarking pipeline enables generation of additional logs as needed. The logs cover a range of solver behaviors, including optimal solutions, infeasible instances, various cutting plane and heuristic strategies, difficult root relaxations, varying optimality gap closures, a wide variety of node counts, and degenerate problems.

Each training example is constructed as a triple:

$$(\text{log_text}, \text{diagnostic_labels}, \text{explanation}) \tag{1}$$

where log_text is the cleaned solver log, diagnostic_labels is a multi-hot vector of bottleneck categories, and explanation is a natural-language summary describing solver behavior and potential improvements. Initial labels are generated using heuristic rules derived from solver metrics (e.g., root gap $> 30\% \rightarrow$ "weak relaxation", node count $> 100{,}000 \rightarrow$ "excessive branching") and then refined through manual expert review.

To validate the heuristic supervision, a stratified subset of 30 solver runs spanning both configurations and the full problem-size range (16 to 345,856 variables) was hand-reviewed against the underlying metric evidence. The expert labels were compared to the heuristic rules, yielding macro-F1 of 0.900 and micro-F1 of 0.929. Recall was 1.0 across all five labels, indicating the rules do not miss positives in the reviewed sample. Precision losses were concentrated on two labels, `degeneracy` (0.71) and `presolve_weakness` (0.50), with all false positives occurring on small instances (fewer than 50 variables) where the rule conditions fired but the underlying phenomenon was not meaningfully present (e.g., presolve has little to reduce on a 30-variable model). This residual noise pattern is documented and treated as a property of the supervision rather than a tuning target, since modifying the rules using the gold subset would compromise its independence as the evaluation instrument for the downstream classifier and diagnostic-generation experiments.

4.3 Bottleneck Classification Model

The classification component is formulated as a multi-label classification problem. Given a solver log, the model predicts a probability for each bottleneck category. The label set includes:

The classification model uses a transformer encoder to generate contextual representations of the log text. Since solver logs can be long, architectures designed for long sequences such as Longformer [6] or Big Bird [7] are candidates.

Label	Indicator
Weak root LP	Large initial optimality gap
Branching difficulty	High node count
Degeneracy	Slow LP pivots
Ineffective cuts	Many rounds, little bound improvement
Presolve weakness	Minimal variable/constraint reduction

4.4 Generative Explanation Model

To produce human-readable diagnostics, a sequence-to-sequence generative model (e.g., T5 or FLAN-T5) generates explanations conditioned on the solver log and predicted diagnostic labels.

Following standard autoregressive sequence modeling used in transformer architectures ([8]), the probability of the generated explanation is factorized into the product of conditional token probabilities:

$$P(\text{explanation} \mid \text{log_text}, \text{labels}) = \prod_{t=1}^{T} P(w_t \mid w_{<t}, \text{log_text}, \text{labels}) \tag{2}$$

where w_t denotes the t-th token of the generated explanation. The model is trained to produce concise summaries describing solver behavior and recommend potential improvements to model formulation or solver configuration.

4.5 LLM Approach Comparison

Three LLM-based approaches will be evaluated:

1. Prompt Engineering: Directly feeding solver logs to a pre-trained LLM with domain-specific prompts. This is the simplest approach and requires no training data.

2. Retrieval-Augmented Generation (RAG): Augmenting the LLM with a database of known solver behavior patterns and diagnostic examples. The system retrieves relevant examples based on log similarity before generating a response.

3. Fine-Tuning: Training a smaller model on the labeled dataset of solver logs and expert annotations. This approach requires the most data but may produce the most consistent domain-specific outputs.

Based on findings from the literature [1], RAG is expected to offer the best balance of accuracy and practicality for this domain, given the relatively small dataset size and the need to reduce hallucination risk.

5 Experimental Plan

[This section will be completed in the final paper.]

The experimental evaluation will assess the system across three dimensions corresponding to the research hypotheses.

5.1 Planned Evaluation Metrics

- Summarization quality (H1): ROUGE scores comparing generated summaries to expert-written reference summaries, supplemented by human evaluation of factual accuracy and completeness.
- Bottleneck detection accuracy (H2): Precision, recall, and F1 score for multi-label classification of bottleneck categories.
- Recommendation quality (H3): Human expert evaluation of generated recommendations assessing usefulness, correctness, and actionability.

5.2 Planned Experiments

1. Baseline comparison: Compare rule-based heuristic diagnostics (current prototype) against LLM-generated diagnostics to quantify the improvement from learned models.
2. Approach comparison: Evaluate prompt engineering, RAG, and fine-tuning approaches on the same test set to determine which strategy is most effective for solver log analysis.

5.3 Planned Statistical Analysis

[Placeholder: Statistical significance tests (e.g., paired t-tests) will be applied to compare approaches. Results will be reported with confidence intervals.]

6 Results

[This section will be completed in the final paper. Preliminary prototype results from rule-based log parsing and heuristic diagnostics are available in the accompanying Jupyter notebook.]

7 Conclusion and Future Work

[This section will be completed in the final paper.]

This project explores the use of large language models to interpret optimization solver logs and provide automated diagnostics for mathematical optimization problems. By combining structured log parsing with LLM reasoning, the Optimization Log Copilot aims to assist both researchers and practitioners in understanding solver behavior and improving optimization performance.

7.1 Limitations

Three limitations of the current work are worth noting. First, the heuristic supervision used to train the bottleneck classifier exhibits a known precision-only noise pattern, with false positives concentrated on the `degeneracy` and `presolve_weakness` labels for instances with fewer than approximately 50 variables. The pattern is documented in Section 4.2 but deliberately not corrected against the hand-reviewed gold subset, since doing so would contaminate that subset's role as the evaluation instrument. Second, the dataset is drawn from a single solver (Rose) and two configurations; behavior of the diagnostics on logs from other solvers such as Gurobi, HiGHS, or CPLEX is not characterized. Third, the diagnostic-quality rubric is applied by a single expert reviewer; inter-annotator agreement was not measured.

7.2 Future Work

Possible future directions include: application to logs from additional solvers such as Gurobi, HiGHS, and CPLEX; interactive solver debugging via a conversational interface; automated parameter tuning recommendations; and integration into solver frameworks.

References

[1] S. Akhtar, S. Khan, and S. Parkinson, "LLM-based event log analysis techniques: A survey," *arXiv preprint arXiv:2502.00677*, 2025. [Online]. Available: `https://arxiv.org/abs/2502.00677`

[2] M. Landauer, S. Onder, F. Skopik, and M. Wurzenberger, "Deep learning for anomaly detection in log data: A survey," *Machine Learning with Applications*, vol. 12, p. 100470, 2023. [Online]. Available: `https://www.sciencedirect.com/science/article/pii/S2666827023000233`

[3] Y. Bengio, A. Lodi, and A. Prouvost, "Machine learning for combinatorial optimization: A methodological tour d'horizon," *arXiv preprint arXiv:1811.06128*, 2018. [Online]. Available: `https://arxiv.org/abs/1811.06128`

[4] J. Kotary, F. Fioretto, P. Van Hentenryck, and B. Wilder, "End-to-end constrained optimization learning: A survey," in *Proceedings of the International Joint Conference on Artificial Intelligence (IJCAI)*, 2021. [Online]. Available: `https://arxiv.org/abs/2103.16378`

[5] M. Yazdani, M. Mostajabdaveh, S. Aref, and Z. Zhou, "EvoCut: strengthening integer programs via evolution-guided language models," *arXiv preprint arXiv:2508.11850*, 2025. [Online]. Available: `https://arxiv.org/abs/2508.11850`

[6] I. Beltagy, M. E. Peters, and A. Cohan, "Longformer: The long-document transformer," *arXiv preprint arXiv:2004.05150*, 2020. [Online]. Available: `https://arxiv.org/abs/2004.05150`

[7] M. Zaheer, G. Guruganesh, A. Dubey, J. Ainslie, C. Alberti, S. Ontañón, P. Pham, Q. Wang, A. Ravula, L. Yang, and A. Ahmed, "Big Bird: Transformers for longer sequences," *arXiv preprint arXiv:2007.14062*, 2020. [Online]. Available: `https://arxiv.org/abs/2007.14062`

[8] C. Raffel, N. Shazeer, A. Roberts, K. Lee, S. Narang, M. Matena, Y. Zhou, W. Li, and P. J. Liu, "Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer," *Journal of Machine Learning Research*, vol. 21, no. 140, pp. 1–67, 2020.